

DEEP LEARNING (23APC3303)

UNIT I: Fundamentals of Neural Networks and Deep Learning

Introduction to Neural Networks and Deep Learning, Biological vs Artificial Neurons, Perceptron and Multilayer Perceptron Activation Functions: Sigmoid, Tanh, ReLU, Leaky ReLU, Forward Propagation and Loss Functions, Gradient Descent, Back propagation, Optimization Algorithms: SGD, Momentum, RMSProp, Adam, Challenges in Deep Learning: Overfitting, Vanishing/Exploding Gradients.

Introduction to Neural Networks and Deep Learning:

Neural Networks

Neural networks are a computational model that shares some properties with the animal brain in which many simple units are working in parallel with no centralized control unit. The weights between the units are the primary means of long-term information storage in neural networks. Updating the weights is the primary way the neural network learns new information. In Chapter 1 we discussed modeling sets of equations in the form of the equation $Ax = b$. In the context of neural networks, the A matrix is still the input data and the b column vector is still the labels or outcomes for each row in the A matrix. The weights on the neural network connections becomes x (the parameter vector). The behavior of neural networks is shaped by its network architecture. A network's architecture can be defined (in part) by the following:

- Number of neurons
- Number of layers
- Types of connections between layers

The most well-known and simplest-to-understand neural network is the feedforward multilayer neural network. It has an input layer, one or many hidden layers, and a single output layer. Each layer can have a different number of neurons and each layer is fully connected to the adjacent layer. The connections between the neurons in the layers form an acyclic graph, as illustrated in Figure 2-1.

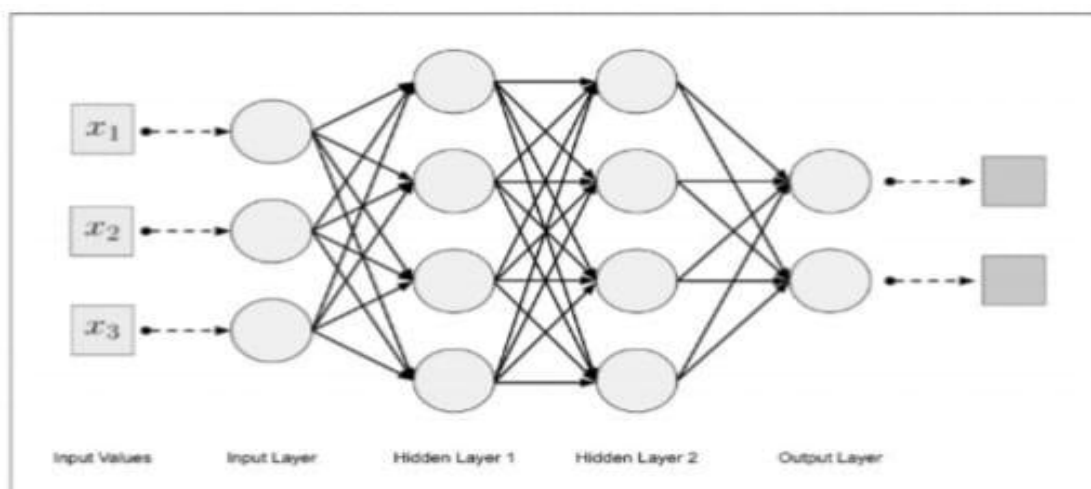


Figure 2-1. Multilayer neural network topology

A feed-forward multilayer neural network can represent any function, given enough artificial neuron units. It is generally trained by a learning algorithm called backpropagation learning. Back

propagation uses gradient descent on the weights of the connections in a neural network to minimize the error on the output of the network.

Local Minima and Back propagation

Back propagation can become stuck in local minima, but in practice it generally performs well. Historically, back propagation has been considered slow, but recent advances in computational power through parallelism and graphics processing units (GPUs) have renewed interest in neural networks.

Much hubris and many words have been transmitted across the internet and in the literature about the connection of neural networks to the human mind. Let's separate some signal from the vast noise and begin with the biological inspiration of artificial neural networks.

The Mechanistic View of the Mind

Rather than constructing a rigid tree that requires all inputs to be one thing or another, we can construct a model that reflects a world sending us partial, ambiguous information, from which we can draw inferences with relative but not total certainty. This aspect of neural networks represents a break from the mechanistic view of the mind, dominant in the early twentieth century, which assumed that our brain interlocked with the world in a deterministic way—like two gears meshing—with clear inputs leading to clear outputs. Now, we assume that, based on incomplete and sometimes contradictory information, humans find ways to plunge forward and act. The human brain infers from probabilities and so do neural networks. We'll provide a brief review of the biological neuron and then take a look at the early precursor to modern neural networks: the perceptron. Building on our understanding of the perceptron, we'll then see how it evolved into the more generalized artificial neuron that supports today's modern feed-forward multilayer perceptron's. The culmination of this chapter will give you, the modern neural network practitioner, the fundamentals to delve further into more exotic deep network architectures.

Deep Learning Examples: Practical Applications in Real Life

Deep learning is a branch of artificial intelligence (AI) that uses algorithms inspired by how the human brain works. It helps computers learn from large amounts of data and make smart decisions. Deep learning is behind many technologies we use every day like voice assistants and medical tools.

This article covers real-world examples of deep learning and explains how it's being used in different fields.

1. Image and Video Recognition

Deep learning has made it possible for machines to understand visual information in ways similar to humans.

- Self-driving cars use deep learning with cameras to detect pedestrians, traffic signs and other vehicles to navigate safely.
- Facial recognition systems match people's facial features for security, phone unlocking or crowd identification.
- Apps use image classification to recognize plants, animals and products making it useful in education and e-commerce.

2. Natural Language Processing (NLP)

NLP allows systems to read, understand and write human language with context and clarity.

- Virtual assistants like Siri and Alexa use NLP to interpret spoken commands and respond naturally.
- Chatbots use NLP to interact with users and answer queries in customer support.
- Text summarization helps create short summaries from long documents, saving time.

3. Speech Recognition

Deep learning has made voice interaction with machines more practical and accurate. It converts speech into text and understands spoken language.

- Voice typing and dictation tools let users speak instead of typing.
- Automated customer support systems respond to voice commands and help users navigate services.
- It is used in virtual meetings and live events for real-time transcription.
- Many smart devices now come with voice control features powered by deep learning.

4. Recommendation Systems

Recommendation engines use deep learning to personalize content and product suggestions. These systems learn from user behavior and improve experiences across platforms.

- Netflix and YouTube suggest videos based on your watch history and preferences.
- E-commerce platforms like Amazon recommend products based on browsing and purchase patterns.
- Music apps suggest playlists and songs that match your taste.

5. Healthcare and Drug Discovery

Deep learning in healthcare helps by speeding up diagnosis and drug development. It assists doctors and researchers in making medical decisions with higher confidence.

- Medical imaging tools detect diseases like cancer from scans such as X-rays and MRIs.
- AI models can predict drug effectiveness by simulating molecular behavior.
- Researchers use it to find potential drug targets faster from biological datasets.
- Deep learning reduces trial-and-error in medicine.

6. Cybersecurity and Scientific Research

Deep learning plays a key role in both securing digital systems and driving scientific discovery. It can detect threats and support faster breakthroughs in research.

- Cybersecurity systems use it to detect unusual activity and prevent hacking or malware attacks.
- Fraud detection models flag suspicious transactions in real time, reducing financial losses.
- It processes massive datasets in fields like physics and material science.

Machine Learning vs Deep Learning

[Machine learning](#) and deep learning are two important branches of artificial intelligence, often used for similar tasks but with different capabilities and approaches. This section offers a simple comparison to help understand where each technique fits best and how they differ in real-world use.

Machine Learning	Deep Learning
Spam email detection	Image recognition
Predictive maintenance	Language translation
Credit scoring	Self-driving cars
Fraud detection	Voice assistants
Customer segmentation	Medical diagnosis
Stock prediction	Robotics

Deep learning is a core part of many technologies we use today. It is making systems smarter and more useful, as technology continues to grow, we expect even more helpful and creative uses of deep learning in the future.

Biological vs Artificial Neurons

The Biological Neuron

The biological neuron is a nerve cell that provides the fundamental functional unit for the nervous systems of all animals. Neurons exist to communicate with one another, and pass electro-chemical impulses across synapses, from one cell to the next, as long as the impulse is strong enough to activate the release of chemicals across a synaptic cleft. The strength of the impulse must surpass a minimum threshold or chemicals will not be released. Figure 2-2 presents the major parts of the nerve cell:

- Soma
- Dendrites
- Axons
- Synapses

The neuron is made up of a nerve cell consisting of a soma (cell body) that has many dendrites but only one axon. The single axon can branch hundreds of times, however. Dendrites are thin structures that arise from the main cell body. Axons are nerve fibers with a special cellular extension that comes from the cell body.

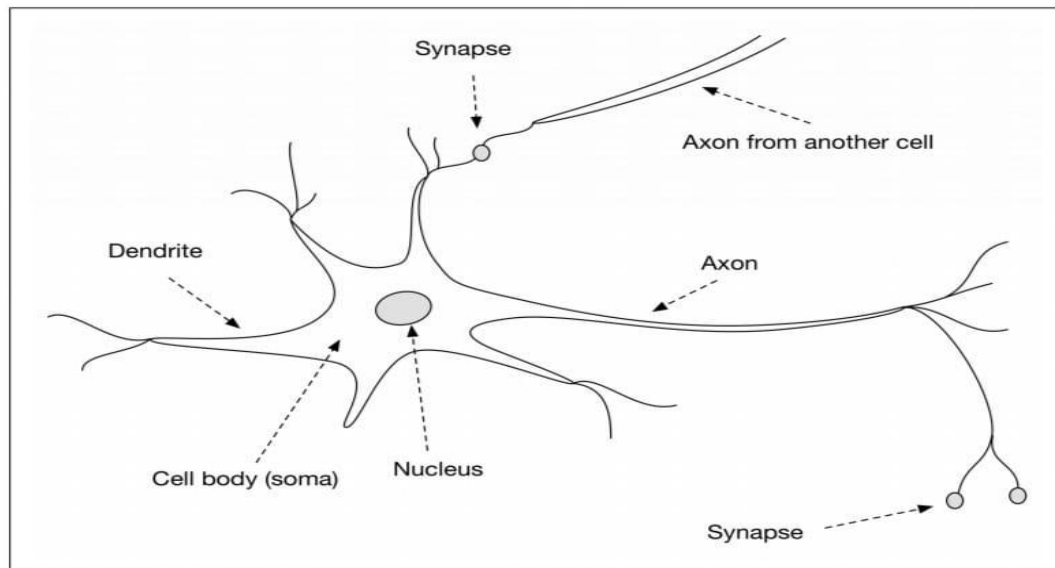


Figure 2-2. The biological neuron

Synapses

Synapses are the connecting junction between axon and dendrites. The majority of synapses send signals from the axon of a neuron to the dendrite of another neuron. The exceptions for this case are when a neuron might lack dendrites, or a neuron lacks an axon, or a synapse, which connects an axon to another axon.

Dendrites

Dendrites have fibers branching out from the soma in a bushy network around the nerve cell. Dendrites allow the cell to receive signals from connected neighbouring neurons and each dendrite is able to perform multiplication by that dendrite's weight value. Here multiplication means an increase or decrease in the ratio of synaptic neurotransmitters to signal chemicals introduced into the dendrite.

Axons

Axons are the single, long fibers extending from the main soma. They stretch out longer distances than dendrites and measure generally 1 centimetre in length (100 times the diameter of the soma). Eventually, the axon will branch and connect to other dendrites. Neurons are able to send electrochemical pulses through cross membrane voltage changes generating action potential. This signal travels along the cell's axon and activates synaptic connections with other neurons.

Information flow across the biological neuron

Synapses that increase the potential are considered excitatory, and those that decrease the potential are considered inhibitory. Plasticity refers the long-term changes in strength of connections in response to input stimulus. Neurons also have been shown to form new connections over time and even migrate. These combined mechanics of connection change drive the learning process in the biological brain.

From biological to artificial

The animal brain has been shown to be responsible for the fundamental components of the mind. We can study the basic components of the brain and understand them. Research has shown ways to map out functionality of the brain and track signals as they move through neurons.

Convolutional Neural Networks and the Mammalian Vision System

Later in the book, we take a look at a deep network called a Convolutional Neural Network (CNN). A CNN's image representation at different layers is similar to how the brain processes visual information. Although this research is interesting, it does not mean that a CNN gives us a full approximation of mammalian brain activity. However, we still do not completely understand how this collection of decentralized functional units provides the foundation for thought and the seat of consciousness.

The Seat of Consciousness

In the eighteenth century, the brain began to be recognized as the "seat of consciousness." By the late nineteenth century, animal brains began to be mapped out to better understand its functional regions. Previous locations of consciousness included the heart and, oddly enough, the spleen. Now that we've established the basics of how a biological neuron works, let's take a look at the first attempts at modeling the neuron with the advent of the perceptron.

Artificial Neurons (Perceptron)

The Perceptron

The perceptron is a linear model used for binary classification. In the field of neural networks, the perceptron is considered an artificial neuron using the Heaviside step function for the activation function, both of which we'll define further later in the chapter. The precursor to the perceptron was the Threshold Logic Unit (TLU) developed by McCulloch and Pitts in 1943, which could

learn the AND and OR logic functions. The perceptron training algorithm is considered a supervised learning algorithm. Both the TLU and the perceptron were inspired by the biological neuron as we'll explore.

History of the perceptron

The perceptron was invented in 1957 at the Cornell Aeronautical Laboratory by Frank Rosenblatt. It was funded by the US Office of Naval Research and was covered by the New York Times: [T]he embryo of an electronic computer that [the Navy] expects will be able to walk, talk, see, write, reproduce itself and be conscious of its existence. Obviously, these predictions were a bit premature—as we've seen many times with the promise of machine learning and AI. Early versions were intended to be implemented as a physical machine rather than a software program. The first software implementation was for the IBM 704, and then later it was implemented in the Mark I Perceptron machine. It also should be noted that McCulloch and Pitts introduced the basic concept of analysing neural activity in 1943 based on thresholds and weighted sums. These concepts were key in developing a model for later variations like the perceptron.

The Mark I Perceptron

The Mark I Perceptron was designed for image recognition for military purposes by the US Navy. The Mark I Perceptron had 400 photocells connected to artificial neurons in the machine, and the weights were implemented by potentiometers. Weight updates were physically performed by electric motors.

Definition of the perceptron

The perceptron is a linear-model binary classifier with a simple input-output relationship as depicted in Figure 2-3, which shows we're summing n number of inputs times their associated weights and then sending this "net input" to a step function with a defined threshold. Typically, with perceptron's, this is a Heaviside step function with a threshold value of 0.5. This function will output a real-valued single binary value (0 or a 1), depending on the input.

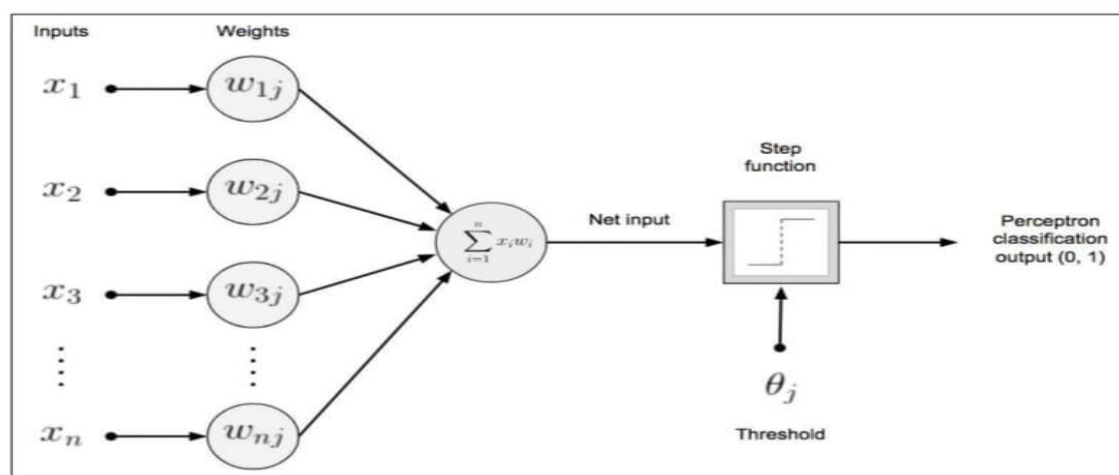


Figure 2-3. Single-layer perceptron

We can model the decision boundary and the classification output in the Heaviside step function equation, as follows: $f(x) = 0$ if $x < 0$, $f(x) = 1$ if $x \geq 0$. To produce the net input to the activation function (here, the Heaviside step function) we take the dot product of the input and the connection weights. We see this summation in the left half of Figure 2-3 as the input to the summation

function. Table 2-1 provides an explanation of how the summation function is performed as well as notes about the parameters involved in the summation function.

Table 2-1. the summation function parameters

Function parameter	Description
$\mathbf{w}$	Vector of real-valued weights on the connections
$\mathbf{w} \cdot \mathbf{x}$	Dot product ($\sum_{i=1}^n w_i x_i$)
n	Number of inputs to the perceptron
b	The bias term (input value does not affect its value; shifts decision boundary away from origin)

The output of the step function (activation function) is the output for the perceptron and gives us a classification of the input values. If the bias value is negative, it forces the learned weights sum to be a much greater value to get a 1 classification output. The bias term in this capacity moves the decision boundary around for the model. Input values do not affect the bias term, but the bias term is learned through the perceptron learning algorithm.

The Single-Layer Perceptron

The perceptron is more widely known as a “single-layer perceptron” in neural network research to distinguish it from its successor the “multilayer perceptron.” As a basic linear classifier, we consider the single-layer perceptron to be the simplest form of the family of feed-forward neural networks.

Relating the Perceptron to the Biological Neuron

Although we don’t have a complete model for how the brain works, we do see how the perceptron was modelled after the biological neuron. In this case, we can see how the perceptron takes input through connections with weights on them in a similar fashion to how synapses pass information to other biological neurons.

The perceptron learning algorithm.

The perceptron learning algorithm changes the weights in the perceptron model until all input records are all correctly classified. The algorithm will not terminate if the learning input is not linearly separable. A linearly separable dataset is one for which we can find the values of a hyperplane that will cleanly divide the two classes of the dataset. The perceptron learning algorithm initializes the weight vector with small random values or 0.0s at the beginning of training. The perceptron learning algorithm takes each input record, as we can see in Figure 2-3, and computes the output classification to check against the actual classification label. To produce the classification, the columns (features) are matched up to weights where n is the number of dimensions in both our input and weights. The first input value is the bias input, which is always 1.0 because we don’t affect the bias input. The first weight is our bias term in this diagram. The dot product of the input vector and the weight vector gives us the input to our activation function, as we’ve previously discussed. If the classification is correct, no weight changes are made. If the classification is incorrect, the weights are adjusted accordingly. Weights are updated between individual training examples in an “online learning” fashion. This loop continues until all of the input examples are correctly classified. If the dataset is not linearly separable, the training

algorithm will not terminate. Figure 2-4 demonstrates a dataset that is not linearly separable, the XOR logic function.

x_0	x_1	y
0	0	0
0	1	1
1	0	1
1	1	0

Figure 2-4. the XOR function

A basic perceptron (single-layer variant) cannot solve the XOR logic modeling problem, illustrating an early limitation of the perceptron model.

Limitations of the early perceptron

After initial promise, the perceptron was found to be limited in the types of patterns it could recognize. The initial inability to solve nonlinear (e.g., datasets that are not linearly separable) problems was seen as a failure for the field of neural networks. The 1969 book *Perceptrons* by Minsky and Paper illustrated the limitations of the singlelayer perceptron. However, what the general industry did not widely realize was that a multilayer perceptron could indeed solve the XOR problem, among many other non- linear problems.

AI Winter I: 1974–1980

The misunderstanding of the multilayer perceptron capabilities was an early public setback that hurt interest in, and funding of, neural networks for the next decade. It wasn't until the resurgence of neural networks in the mid-1980s that backpropagation became popular (although backpropagation was originally discovered in 1974 by Webos) and neural networks enjoyed a second wave of interest.

Multilayer Feed-Forward Networks

The multilayer feed-forward network is a neural network with an input layer, one or more hidden layers, and an output layer. Each layer has one or more artificial neurons. These artificial neurons are similar to their perceptron precursor yet have a different activation function depending on the layer's specific purpose in the network. We'll look more closely at the layer types in multilayer perceptron later in the chapter. For now, let's look more closely at this evolved artificial neuron that emerged from the limitations of the single-layer perceptron.

Evolution of the artificial neuron

The artificial neuron of the multilayer perceptron is similar to its predecessor, the perceptron, but it adds flexibility in the type of activation layer we can use. Figure 2-5 shows an updated diagram for the artificial neuron that is based on the perceptron.

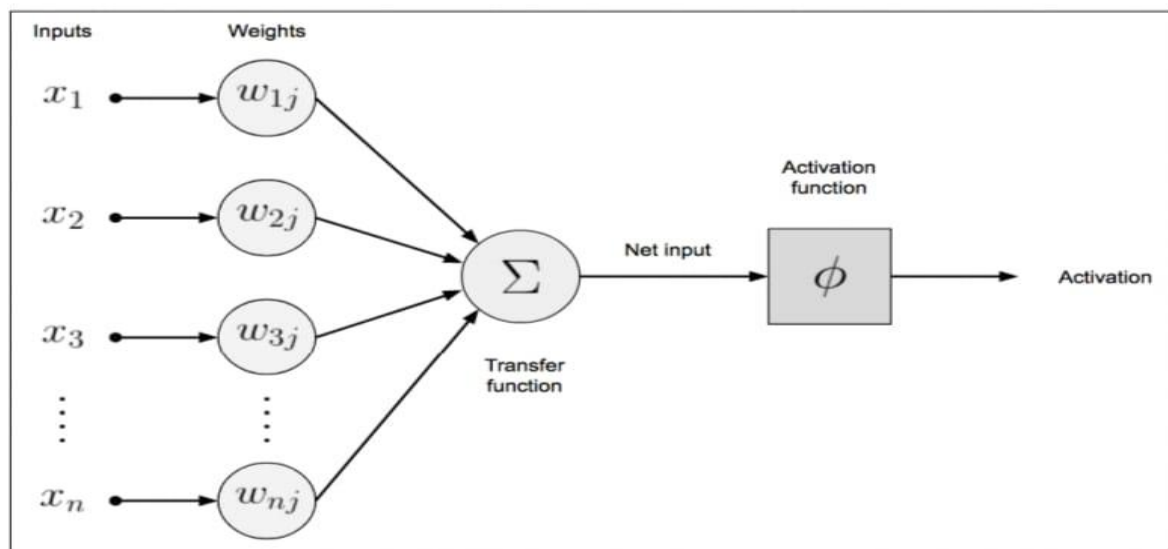


Figure 2-5. Artificial neuron for a multilayer perceptron

This diagram is similar to Figure 2-3 for the single-layer perceptron, yet we notice a more generalized activation function. We'll develop this diagram further in a detailed look at the artificial neuron as we proceed.

A Note about the Term “Neuron”

From this point throughout the remainder of the book, when we use the term “neuron” we’re referring to the artificial neuron based on Figure 2-5.

The net input to the activation function is still the dot product of the weights and input features yet the flexible activation function allows us to create different types out of output values. This is a major contrast to the earlier perceptron design that used a piecewise linear Heaviside step function as this improvement now allowed the artificial neuron because express more complex activation output. Artificial neuron input. The artificial neuron (see Figure 2-6) takes input that, based on the weights on the connections, can be ignored (by a 0.0 weight on an input connection) or passed on to the activation function. The activation function also has the ability to filter out data if it does not provide a non-zero activation value as output.

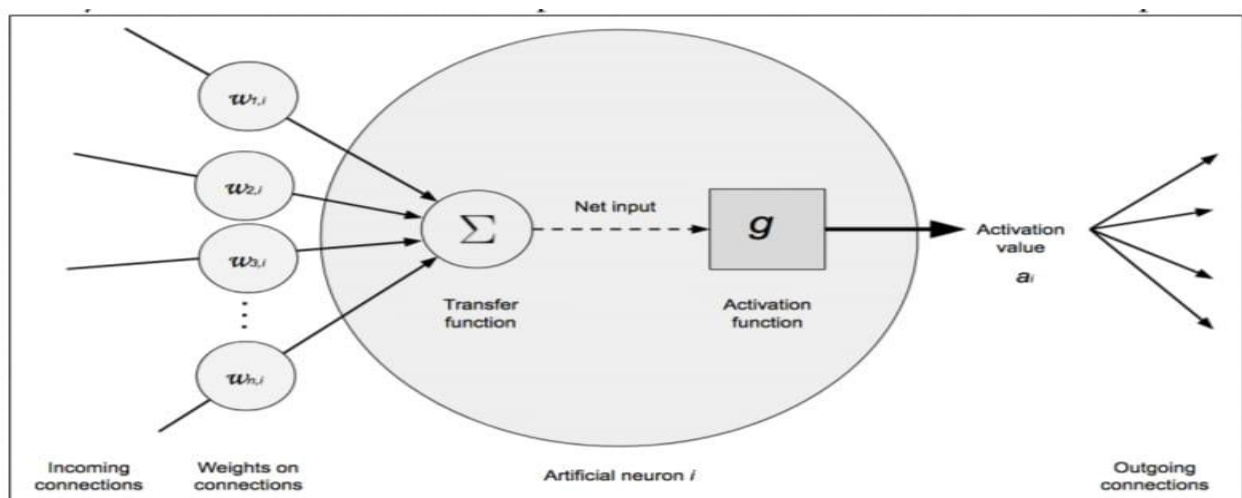


Figure 2-6. Details of an artificial neuron in a multilayer perceptron neural network

We express the net input to a neuron as the weights on connections multiplied by activation incoming on connection, as shown in Figure 2-6. For the input layer, we're just taking the feature at that specific index, and the activation function is linear (it passes on the feature value). For hidden layers, the input is the activation from other neurons. Mathematically, we can express the net input (total weighted input) of the artificial neuron as

$$\text{input_sum}_i = W_i \cdot A_i$$

where W_i is the vector of all weights leading into neuron i and A_i is the vector of activation values for the inputs to neuron i . Let's build on this equation by accounting for the bias term that is added per layer (explained further below):

$$\text{input_sum}_i = W_i \cdot A_i + b$$

To produce output from the neuron, we'd then wrap this net input with an activation function g , as demonstrated in the following equation:

$$a_i = g(\text{input_sum}_i)$$

We can then expand this function with the definition of input_i :

$$a_i = g(W_i \cdot A_i + b)$$

This activation value for neuron i is the output value passed on to the next layer through connections to other artificial neurons (multiplied by weights on connections) as an input value.

Different Notations

An alternate notation for expressing the output of an artificial neuron we often see in research papers is as follows:

$$h_{w,b}(x) = g(w \cdot x + b)$$

This notation is slightly different from the previous equation above. We show this alternate notation to illustrate how some papers will use slightly different notations to explain these concepts, and we want you to be able to recognize the variants.

If our activation function is the sigmoid function, we'll have this:

$$g(z) = 1 / (1 + e^{-z})$$

This output will have the range $[0, 1]$, which is the same output as the logistic regression function. Sigmoid Activation Functions in Practice We list the sigmoid activation function here for historical demonstration. As we'll see later on in this book, the sigmoid activation function has largely fallen out of favour.

The inputs are the data from which you want to produce information, and the connection weights and biases are the quantities that govern the activity, activating it or not. As with the perceptron, there is a learning algorithm to change the weights and bias value for each artificial neuron. During the training phase, the weights and biases change as the network learns. We'll cover the learning algorithm for neural networks later in the chapter.

Just as biological neurons don't pass on every electro-chemical impulse they receive, artificial neurons are not just wires or diodes passing on a signal. They are designed to be selective. They

filter the data they receive, and aggregate, convert, and transmit only certain information to the next neuron(s) in the network. As these filters and transformations work on data, they convert raw input data to useful information in the context of the larger multilayer perceptron neural network. We illustrate this effect more in the next section.

Artificial neurons can be defined by the kind of input they are able to receive (binary or continuous) and the kind of transform (activation function) they use to produce output. In DL4J, all neurons in a layer have the same activation function.

Connection weights

Weights on connections in a neural network are coefficients that scale (amplify or minimize) the input signal to a given neuron in the network. In common representations of neural networks, these are the lines/arrows going from one point to another, the edges of the mathematical graph. Often, connections are notated as w in mathematical representations of neural networks.

Biases

Biases are scalar values added to the input to ensure that at least a few nodes per layer are activated regardless of signal strength. Biases allow learning to happen by giving the network action in the event of low signal. They allow the network to try new interpretations or behaviors. Biases are generally notated b , and, like weights, biases are modified throughout the learning process.

Activation functions

The functions that govern the artificial neuron's behavior are called activation functions. The transmission of that input is known as forward propagation. Activation functions transform the combination of inputs, weights, and biases. Products of these transforms are input for the next node layer. Many (but not all) nonlinear transforms used in neural networks transform the data into a convenient range, such as 0 to 1 or -1 to 1. When an artificial neuron passes on a nonzero value to another artificial neuron, it is said to be activated.

Activations

Activations are the values passed on to the next layer from each previous layer. These values are the output of the activation function of each artificial neuron. In “Activation Functions” on page 65 we'll review the different types of activation functions and their general function in the broader context of neural networks.

Activation Functions and Their Importance

Activation functions and their usage will be a continuing theme throughout almost every chapter for the remainder of this book. The DL4J library uses a layer-based architecture revolving around different types of activation functions.

Comparing the biological neuron and the artificial neuron

If we loop back for a moment and think about the biological neuron on which the artificial neuron is based, we can ask, “How close does the artificial variant match up to the biological version?” The concepts match up with the input connection functionality being performed by dendrites in

the biological neuron and the summation functionality being provided by the soma. Finally, we see the activation function being performed by the axon in the biological neuron.

Limitations of Comparisons

We note (again) that the biological neuron is still more complex than the artificial variant. Research continues toward a better understanding of the biological neuron's function.

Forward Propagation

Feed-forward neural network architecture

Now that we understand the differences between the artificial neuron and the perceptron, we can better understand the structure of the full multilayer feed-forward neural network. With multilayer feed-forward neural networks, we have artificial neurons arranged into groups called layers. Building on the layer concept, we see that the multilayer neural network has the following:

- A single input layer
- One or many hidden layers, fully connected
- A single output layer

As Figure 2-7 depicts, the neurons in each layer (represented by the circles) are all fully connected to all neurons in all adjacent layers. The neurons in each layer all use the same type of activation function (most of the time). For the input layer, the input is the raw vector input. The input to neurons of the other layers is the output (activation) of the previous layer's neurons. As data moves through the network in a feed-forward fashion, it is influenced by the connection weights and the activation function type. Let's now take a look at the specifics of each layer type.

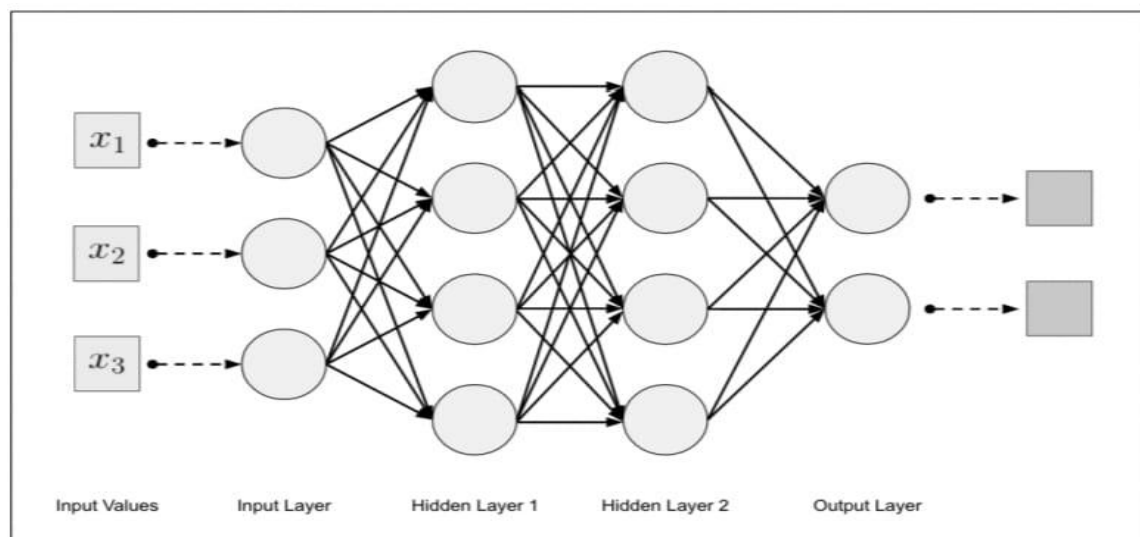


Figure 2-7. Fully connected multilayer feed-forward neural network topology

Input layer.

This layer is how we get input data (vectors) fed into our network. The number of neurons in an input layer is typically the same number as the input feature to the network. Input layers are followed by one or more hidden layers (explained in the next section). Input layers in classical

feed-forward neural networks are fully connected to the next hidden layer, yet in other network architectures, the input layer might not be fully connected.

Hidden layer.

There are one or more hidden layers in a feed-forward neural network. The weight values on the connections between the layers are how neural networks encode the learned information extracted from the raw training data. Hidden layers are the key to allowing neural networks to model nonlinear functions, as we saw from the limitations of the single-layer perceptron networks.

Output layer.

We get the answer or prediction from our model from the output layer. Given that we are mapping an input space to an output space with the neural network model, the output layer gives us an output based on the input from the input layer. Depending on the setup of the neural network, the final output may be a real valued output (regression) or a set of probabilities (classification). This is controlled by the type of activation function we use on the neurons in the output layer. The output layer typically uses either a SoftMax or sigmoid activation function for classification. We'll discuss the difference between these two types of activation functions later in the chapter.

Connections between layers.

In a fully connected feed-forward network, the connections between layers are the outgoing connections from all neurons in the previous layer to all of the neurons in the next layer. We change these weights progressively as our algorithm finds the best solution it can with the back propagation learning algorithm. We can understand the weights mathematically by thinking of them as the parameter vector in the earlier linear algebra section describing the machine learning process as optimizing the parameter vector (e.g., “weights” here) to minimize error. We've now covered the basic structure of feed-forward neural networks. The rest of this chapter provides a more detailed look at the training mechanics of back propagation as well as specifics of activation functions. The chapter closes with a review of common loss functions and hyper parameters.

Training Neural Networks

A well-trained artificial neural network has weights that amplify the signal and dampen the noise. A bigger weight signifies a tighter correlation between a signal and the network's outcome. Inputs paired with large weights will affect the network's interpretation of the data more than inputs paired with smaller weights. The process of learning for any learning algorithm using weights is the process of readjusting the weights and biases, making some smaller and others larger, thereby allocating significance to certain bits of information and minimizing other bits. This helps our model learn which predictors (or features) are tied to which outcomes, and adjusts the weights and biases accordingly. In most datasets, certain features are strongly correlated with certain labels (e.g., square footage relates to the sale price of a house). Neural networks learn these relationships blindly by making a guess based on the inputs and weights and then measuring how accurate the results are. The loss functions in optimization algorithms, such as stochastic gradient descent (SGD), reward the network for good guesses and penalize it for bad ones. SGD moves the parameters of the network toward making good predictions and away from bad ones. Another way to look at the learning process is to view labels as theories and the feature set as evidence. Then, we can make the analogy that the network seeks to establish the correlation between the theory and the evidence. The model attempts to answer the question “which theory does the evidence

support?” With these ideas in mind, let’s take a look at the learning algorithm most commonly associated with neural networks: back propagation learning.

Activation Functions:

We use activation functions to propagate the output of one layer’s nodes forward to the next layer (up to and including the output layer). Activation functions are a scalar-to-scalar function, yielding the neuron’s activation. We use activation functions for hidden neurons in a neural network to introduce nonlinearity into the network’s modeling capabilities. Many activation functions belong to a logistic class of trans- forms that (when graphed) resemble an S. This class of function is called sigmoidal. The sigmoid family of functions contains several variations, one of which is known as the Sigmoid function. Let’s now take a look at some useful activation functions in neural networks.

Types of Activation Functions:

- **Linear**
- **Non- Linear**

Linear Activation Functions

A linear transform (see Figure 2-11) is basically the identity function, and $f(x) = Wx$, where the dependent variable has a direct, proportional relationship with the inde- pendent variable. In practical terms, it means the function passes the signal through unchanged.

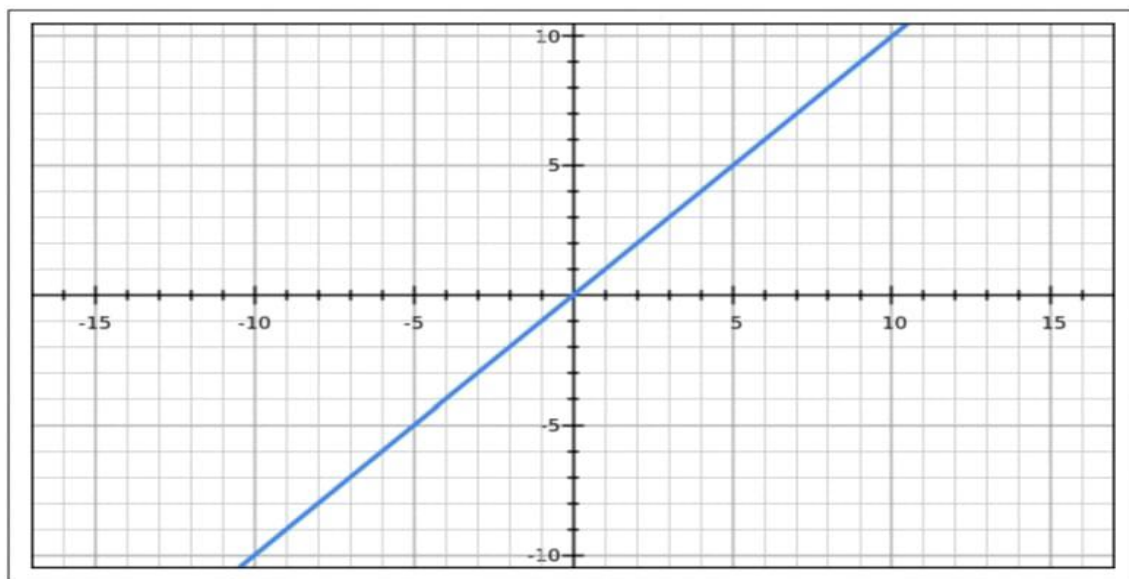


Figure 2-11. Linear activation function

We see this activation function used in the input layer of neural networks.

Non- Linear Activation Functions

Sigmoid

Like all logistic transforms, sigmoids can reduce extreme values or outliers in data without removing them. The vertical line in Figure 2-12 is the decision boundary.

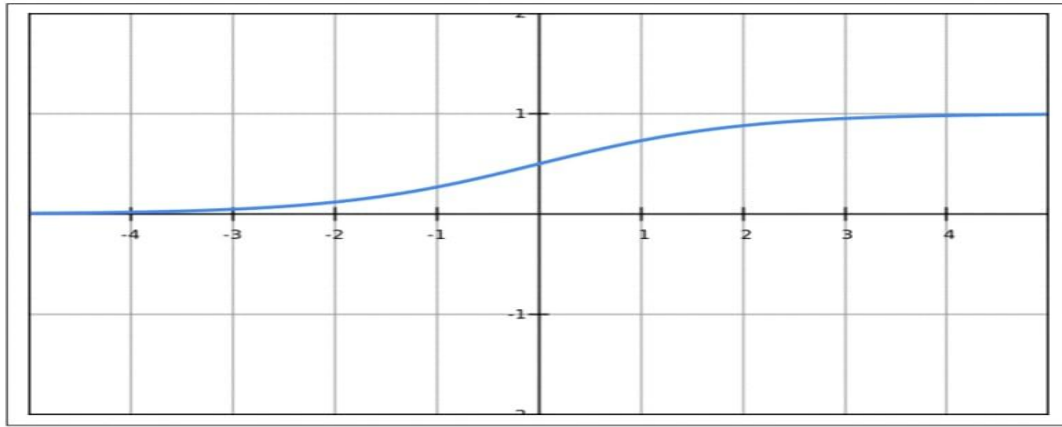


Figure 2-12. Sigmoid activation function

A sigmoid function is a machine that converts independent variables of near infinite range into simple probabilities between 0 and 1, and most of its output will be very close to 0 or 1.

Understanding Sigmoid Output

A sigmoid activation function outputs an independent probability for each class.

Tanh

Pronounced “tanch,” tanh is a hyperbolic trigonometric function (see Figure 2-13). Just as the tangent represents a ratio between the opposite and adjacent sides of a right triangle, tanh represents the ratio of the hyperbolic sine to the hyperbolic cosine: $\tanh(x) = \sinh(x) / \cosh(x)$. Unlike the Sigmoid function, the normalized range of tanh is -1 to 1 . The advantage of tanh is that it can deal more easily with negative numbers.

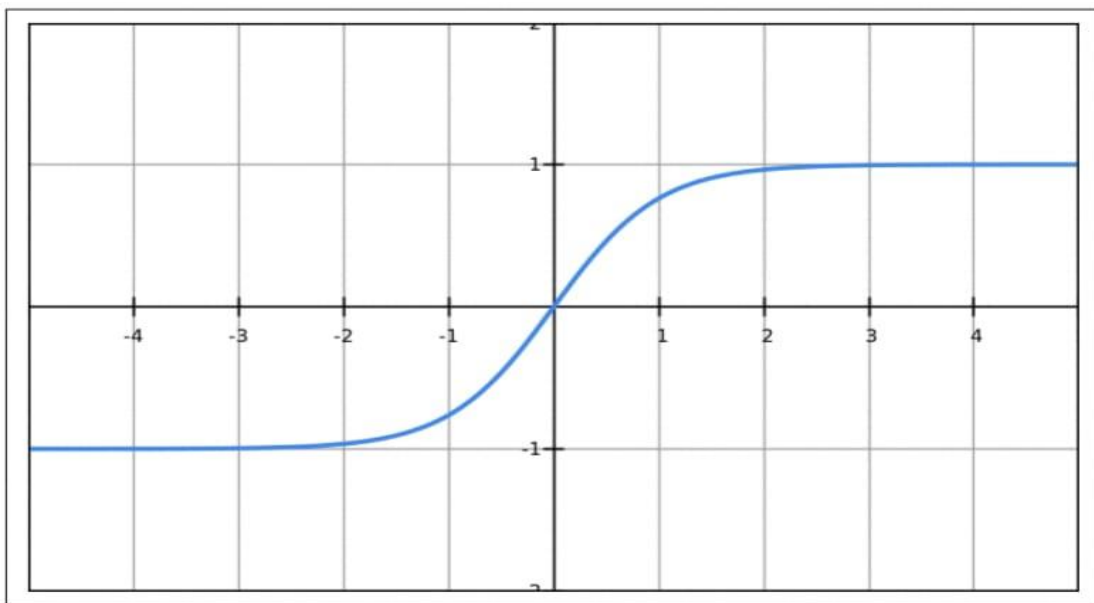


Figure 2-13. Tanh activation function

Hard Tanh

Similar to tanh, hard tanh simply applies hard caps to the normalized range. Any- thing more than 1 is made into 1, and anything less than -1 is made into -1 . This allows for a more robust activation function that allows for a limited decision bound- ary.

Softmax

Softmax is a generalization of logistic regression inasmuch as it can be applied to continuous data (rather than classifying binary) and can contain multiple decision boundaries. It handles multinomial labeling systems. Softmax is the function you will often find at the output layer of a classifier.

Understanding Softmax Output

The softmax activation function returns the probability distribution over mutually exclusive output classes. To further illustrate the idea of the softmax output layer and how to use it, let's consider two use cases. If we have a multiclass modeling problem yet we care only about the best score across these classes, we'd use a softmax output layer with an `argmax()` function to get the highest score of all the classes.

Dealing with Multiple Classifications

If we want to get multiple classifications per output (e.g., "person + car"), we do not want softmax as an output layer. Instead, we'd use the sigmoid output layer giving us a probability for every class independently. For the case in which we have a large set of labels (e.g., thousands of labels), we'd use the variant of the softmax activation function called the hierarchical softmax activation function. This variant decomposes the labels into a tree structure, and the softmax classifier is trained at each node of the tree to direct the branching for classification.

Rectified Linear

Rectified linear is a more interesting transform that activates a node only if the input is above a certain quantity. While the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable $f(x) = \max(0, x)$, as demonstrated in Figure 2-14.

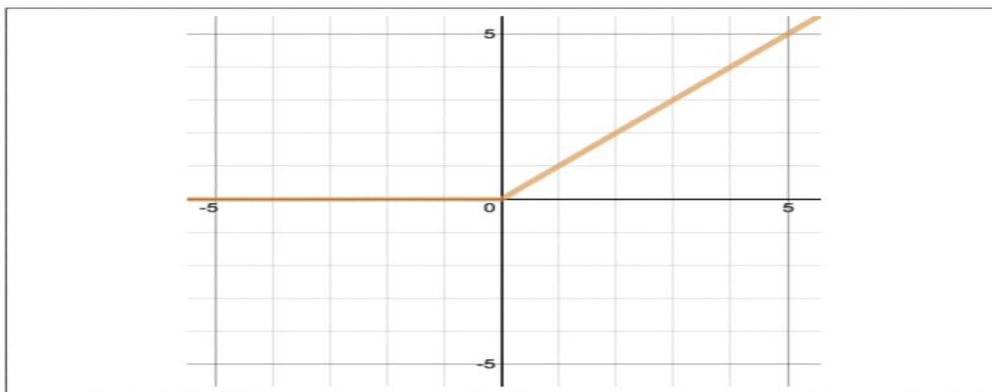


Figure 2-14. Rectified linear activation function

Rectified linear units (ReLU) are the current state of the art because they have proven to work in many different situations. Because the gradient of a ReLU is either zero or a constant, it is possible to reign in the vanishing exploding gradient issue. ReLU activation functions have shown to train better in practice than sigmoid activation functions.

The Unreasonable Effectiveness of ReLU Activation Functions

Compared to the sigmoid and tanh activation functions, the ReLU activation function does not suffer from vanishing gradient issues. If we use hard max as the activation function, we can

induce sparsity in the activation output from the layer. Research has shown deep networks using ReLU activation functions to train well without using pretraining techniques.

Leaky ReLU

Leaky ReLUs are a strategy to mitigate the “dying ReLU” issue.² As opposed to having the function being zero when $x < 0$, the leaky ReLU will instead have a small negative slope (e.g., “around 0.01”). Some success has been seen in practice with this ReLU variation but results are not always consistent. The equation is given here:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{otherwise} \end{cases}$$

Softplus

This activation function is considered to be the “smooth version of the ReLU,” as is illustrated in Figure 2-15. Compare this plot to the ReLU in Figure 2-14. Figure 2-15 shows that the softplus activation function ($f(x) = \ln[1 + \exp(x)]$) has a similar shape to the ReLU. We also notice the differentiability and nonzero derivative of the softplus everywhere on the graph, in contrast to the ReLU.

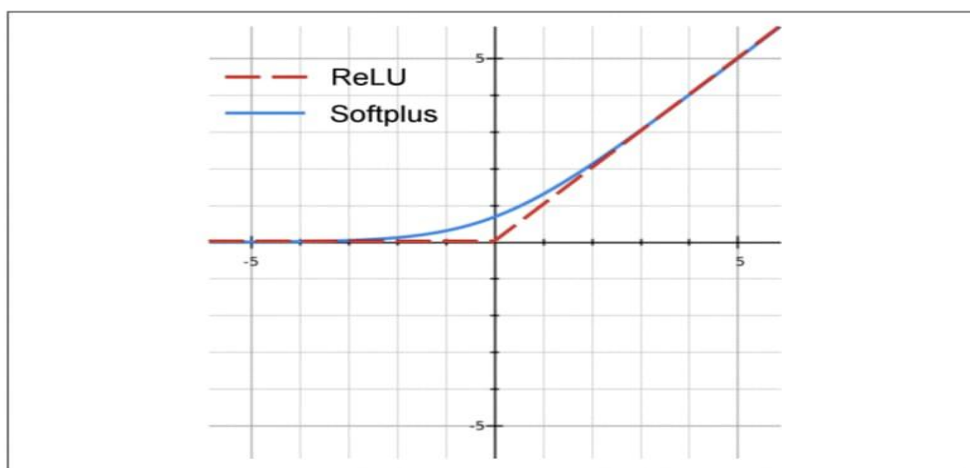


Figure 2-15. Visualizing the ReLU and sotplus activation functions

Activation functions in layers

Activation functions are applied to nodes in a neural network to introduce non-linearity, allowing the network to learn complex patterns. Different layers typically use different activation functions based on their roles:

1. Input Layer

- **Activation Function: None (or Linear)**
 - The input layer generally **does not apply an activation function**, as it simply feeds the raw input data into the network.
 - In some frameworks, it's treated as having a **linear (identity)** function:

$$f(x) = x$$

2. Hidden Layers

- Common Activation Functions:.

Activation	Formula	Properties
ReLU (Rectified	$f(x) = \max(0, x)$	Most common; efficient and avoids

Activation	Formula	Properties
Linear Unit)		vanishing gradient
Leaky ReLU	$f(x) = x \text{ if } x > 0 \text{ else } \alpha x$ (α small, e.g. 0.01)	Fixes ReLU's "dying neuron" issue
Sigmoid	$f(x) = 1 / (1 + e^{(-x)})$	Historically used; causes vanishing gradients
Tanh	$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$	Like sigmoid but zero-centered
ELU, Swish, GELU	Various modern alternatives for smoother gradients	Used in deeper or specialized networks

- **ReLU** is the default for most feedforward networks today due to its simplicity and performance.

3. Output Layer

- Depends on the **task type**:

Task Type	Activation Function	Output Example
Binary Classification	Sigmoid	Output: value between 0 and 1 (probability)
Multi-Class Classification	Softmax	Output: probability distribution across classes
Multi-Label Classification	Sigmoid (per output node)	Independent probabilities for each label
Regression (Unbounded)	Linear (None)	Output: real number
Regression (bounded)	Tanh or Sigmoid	Output: bounded range (e.g. [0,1] or [-1,1])

Summary table:

Layer Type	Common Activation
Input Layer	None / Linear
Hidden Layers	ReLU, Leaky ReLU, Tanh
Output Layer	Sigmoid / Softmax / Linear (depends on task)

Loss Functions:

A **loss function** is a mathematical way to measure how good or bad a model's predictions are compared to the actual results.

It gives a **single number** that tells us how far off the predictions are.

- The **smaller the number**, the better the model is doing.
- Loss functions are used to **train models**.

Why are Loss Functions Important?

- Guide Model Training:** During training, algorithms such as *Gradient Descent* use the loss function to adjust the model's parameters and reduce error.
- Measure Performance:** Helps evaluate how well predictions match actual values.
- Affect Learning Behavior:** Different loss functions make the model focus on different types of mistakes.

Regression Loss Functions

Used when the model predicts **continuous values** (e.g., house price, temperature, age).

1.1 Mean Squared Error (MSE) Loss

Mean Squared Error (MSE) Loss is one of the most widely used loss functions for regression tasks. It calculates the average of the squared differences between the predicted values and the actual values. It is simple to understand and sensitive to outliers because the errors are squared which can affect the loss.

Formula:

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

1.2 Mean Absolute Error (MAE) Loss

Mean Absolute Error (MAE) Loss is another commonly used loss function for regression. It calculates the average of the absolute differences between the predicted values and the actual values. It is less sensitive to outliers compared to MSE. But it is not differentiable at zero which can cause issues for some optimization algorithms.

Formula:

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

1.3 Huber Loss

Huber Loss combines the advantages of MSE and MAE. It is less sensitive to outliers than MSE and differentiable everywhere unlike MAE. It requires tuning of the parameter δ . Huber Loss is defined as:

Formula:

$$L_{\delta}(y, \hat{y}) = \begin{cases} 0.5 (y_i - \hat{y}_i)^2 & \text{if } |y_i - \hat{y}_i| \leq \delta \\ \delta |y_i - \hat{y}_i| - 0.5 \delta^2 & \text{if } |y_i - \hat{y}_i| > \delta \end{cases}$$

2. Classification Loss Functions

Classification loss functions are used to evaluate how well a classification model's predictions match the actual class labels. There are different types of classification Loss functions:

2.1. Binary Cross-Entropy Loss (Log Loss)

Binary Cross-Entropy Loss is also known as Log Loss and is used for binary classification problems. It measures the performance of a classification model whose output is a probability value between 0 and 1.

$$\text{BCE} = -(1/n) \sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Where:

- n is the number of data points
- y_i is the actual binary label (0 or 1)

- $\hat{y}_i$ is the predicted probability.

2.2. Categorical Cross-Entropy Loss

Categorical Cross-Entropy Loss is used for multiclass classification problems. It measures the performance of a classification model whose output is a probability distribution over multiple classes.

$$\text{CCE} = - \sum_i \sum_j y_{ij} \log(\hat{y}_{ij})$$

Where:

- n is the number of data points
- k is the number of classes,
- y_{ij} is the binary indicator (0 or 1) if class label j is the correct classification for data point i
- $\hat{y}_{ij}$ is the predicted probability for class j .

2.3. Hinge Loss

Hinge Loss is used for training classifiers especially for support vector machines (SVMs). It is suitable for binary classification tasks as it is not differentiable at zero.

$$\text{Hinge} = (1/n) \sum \max(0, 1 - y_i * \hat{y}_i)$$

Where:

- y_i is the actual label (-1 or 1)
- $\hat{y}_i$ is the predicted value.

Summary Table:

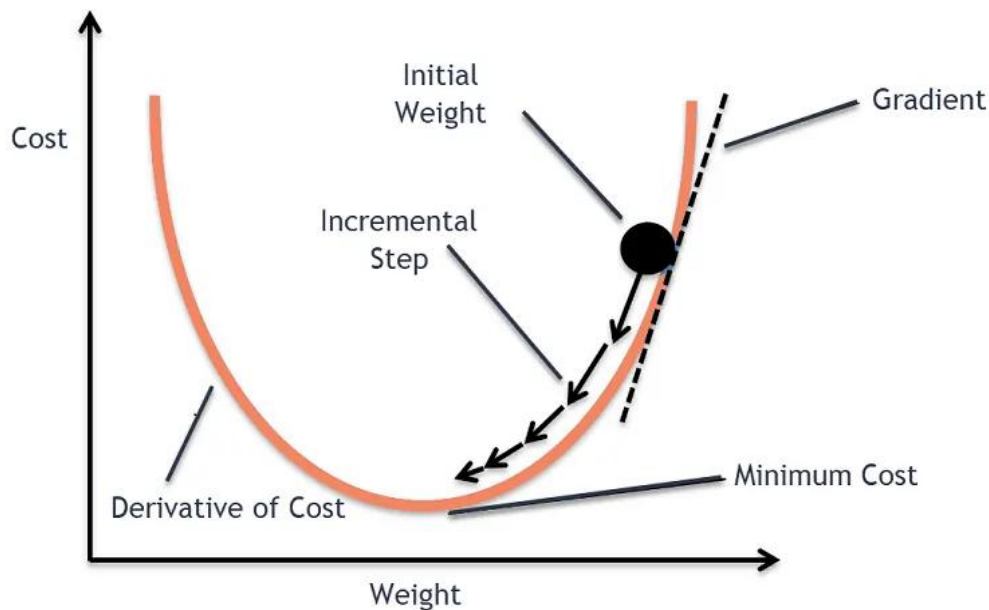
Category	Loss Function	Best For	Key Property
Regression	MSE	Continuous predictions	Sensitive to outliers
Regression	MAE	Continuous predictions	Robust to outliers
Regression	Huber	Continuous predictions	Mix of MSE & MAE
Classification	Binary Cross-Entropy	Binary tasks	Works with probabilities
Classification	Categorical Cross-Entropy	Multi-class tasks	One-hot labels
Classification	Sparse CCE	Multi-class tasks	Integer labels
Classification	KL Divergence	Probabilistic models	Compares distributions
Classification	Hinge Loss	SVMs	Margin-based loss

Optimization Algorithms:

Optimization algorithms are methods used to minimize the loss function during training of a deep learning model. They adjust the model parameters (weights and biases) step by step so that predictions get closer to actual outputs.

1. Gradient Descent (GD)

Gradient Descent is the foundation of most optimization algorithms. It updates the model parameters in the opposite direction of the gradient of the loss function.



How Does Gradient Descent Work?

- The algorithm optimizes to minimize the model's cost function.
- The cost function measures how well the model fits the training data and defines the difference between the predicted and actual values.
- The cost function's gradient is the derivative with respect to the model's parameters and points in the direction of the steepest ascent.
- The algorithm starts with an initial set of parameters and updates them in small steps to minimize the cost function.
- In each iteration of the algorithm, it computes the gradient of the cost function with respect to each parameter.
- The gradient tells us the direction of the steepest ascent, and by moving in the opposite direction, we can find the direction of the steepest descent.
- The learning rate controls the step size, which determines how quickly the algorithm moves towards the minimum.
- The process is repeated until the cost function converges to a minimum. Therefore indicating that the model has reached the optimal set of parameters.
- Different variations of gradient descent include batch gradient descent, stochastic gradient descent, and mini-batch gradient descent, each with advantages and limitations.
- Efficient implementation of gradient descent is essential for performing well in machine learning tasks. The choice of the learning rate and the number of iterations can significantly impact the algorithm's performance.

General Update Rule:

$$\theta = \theta - \eta \nabla_{\theta} L(\theta)$$

Where:

- $\theta \rightarrow$ model parameters (weights, biases)
- $\eta \rightarrow$ learning rate (step size)
- $L(\theta) \rightarrow$ loss function
- $\nabla_{\theta} L(\theta) \rightarrow$ gradient of loss with respect to θ

Types of Gradient Descent

1. Batch Gradient Descent (BGD)

- Uses the **entire training dataset** for each update.
- Formula:

$$\theta = \theta - \eta \left(\frac{1}{N} \right) \sum \nabla_{\theta} L(x_i, y_i)$$

- Pros: Stable, accurate direction.
- Cons: Very slow, high memory usage.
- Use case: Small datasets.

2. Stochastic Gradient Descent (SGD)

- Updates parameters **using only one sample** at a time.
- Formula:

$$\theta = \theta - \eta \nabla_{\theta} L(x_i, y_i)$$

- Pros: Fast, can escape local minima.
- Cons: Very noisy updates.
- Use case: Streaming/online learning.

3. Mini-Batch Gradient Descent (MBGD)

- Uses a **small group (batch)** of samples for each update.
- Formula:

$$\theta = \theta - \eta \left(\frac{1}{m} \right) \sum \nabla_{\theta} L(x_i, y_i)$$

- Pros: Best balance (fast + stable), efficient on GPUs.
- Cons: Needs batch size tuning.
- Use case: Standard in deep learning (default choice).

Comparison Table

Type	Data Used per Update	Speed	Stability	Memory Need	Usage
Batch GD	Whole dataset	Slow	Stable	High	Small datasets
Stochastic GD	1 sample	Very fast	Noisy	Low	Online learning
Mini-Batch GD	Small group (e.g. 32, 64)	Moderate	Balanced	Medium	Deep Learning

2. Momentum

- Idea: Accelerates SGD by considering the **past update direction**.
- Update Rule:

$$\begin{aligned} \mathbf{v} &= \beta \mathbf{v} + \eta \nabla_{\theta} L(\theta) \\ \theta &= \theta - \mathbf{v} \end{aligned}$$

Where β is momentum coefficient (e.g., 0.9).

3. Adagrad

- Adapts learning rate for each parameter individually.
- Formula:

$$\theta = \theta - (\eta / \sqrt{G + \epsilon}) \nabla_{\theta} L(\theta)$$

where G is sum of squared past gradients.

4. RMSProp

- Fixes Adagrad's decreasing learning rate problem using **exponential moving average**.
- Formula:

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta) g_t^2$$
$$\theta = \theta - (\eta / \sqrt{E[g^2]_t + \epsilon}) g_t$$

6. Adam (Adaptive Moment Estimation)

- Combines **Momentum + RMSProp**.
- Most widely used optimizer in Deep Learning.
- Formula:

$$m = \beta_1 m + (1 - \beta_1) g$$
$$v = \beta_2 v + (1 - \beta_2) g^2$$
$$\theta = \theta - (\eta * m / (\sqrt{v} + \epsilon))$$

7. AdaMax

- Variant of Adam that uses **infinity norm**.
- More stable in some cases.

8. Nadam (Nesterov + Adam)

- Combines **Nesterov momentum** with Adam.
- Often achieves faster convergence.

Key Points:

- **Gradient Descent (Batch, Stochastic, Mini-Batch)** is the foundation.
- **Mini-Batch GD** is the standard in deep learning.
- **Momentum** help speed up training.
- **Adagrad, RMSProp, Adam** handle adaptive learning rates.
- **Adam** is the default choice for most deep learning models.

Back propagation

Back propagation, short for "backward propagation of errors," is a fundamental algorithm used to train artificial neural networks, particularly in the context of deep learning. Its primary purpose is to efficiently adjust the weights and biases within a neural network to minimize the difference

between the network's predicted output and the actual target output. This difference is quantified by a "loss function" or "error function."

The process of back propagation involves two main phases:

Forward Pass:

During this phase, input data is fed through the neural network, layer by layer, from the input layer to the output layer. Each neuron performs a weighted sum of its inputs and applies an activation function to produce its output, which then serves as input for the next layer. This continues until the network produces a final output prediction.

Backward Pass (Error Propagation):

After the forward pass, the network's predicted output is compared to the true target output, and the error (or loss) is calculated. This error is then propagated backward through the network, from the output layer towards the input layer. Using the chain rule from calculus, back propagation calculates the gradient of the loss function with respect to each weight and bias in the network. This gradient indicates how much each parameter contributes to the overall error.

Weight and Bias Updates:

Finally, these calculated gradients are used to update the weights and biases of the network using an optimization algorithm, most commonly gradient descent or its variants (e.g., Adam, RMSprop). The updates are made in a direction that reduces the loss, effectively teaching the network to make more accurate predictions in subsequent iterations.

In essence, back propagation allows neural networks to "learn from their mistakes" by systematically adjusting their internal parameters based on the errors they make during the training process. This iterative adjustment process continues until the network achieves a desired level of performance or convergence.

Back Propagation Algorithm

BACKPROPAGATION (training_example, η , n_{in} , n_{out} , n_{hidden})

- Each training example is a pair of the form (x, t) , where (x) is the vector of network input values, and (t) is the vector of target network output values.
- η is the learning rate (e.g., 0.05).
- n_i is the number of network inputs,
- n_{hidden} the number of units in the hidden layer, and
- n_{out} the number of output units.
- The input from unit i into unit j is denoted x_{ji} , and the weight from unit i to unit j is denoted w_{ji}

Back Propagation Algorithm

- Create a feed-forward network with n_i inputs, n_{hidden} hidden units, and n_{out} output units.
- Initialize all network weights to small random numbers
- Until the termination condition is met, Do
 - For each (x, t) , in training examples, Do
 - Propagate the input forward through the network:
 1. Input the instance x , to the network and compute the output o_u of every unit u in the network.
 - Propagate the errors backward through the network
 2. For each network unit k , calculate its error term δ_k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k)$$

3. For each network unit h , calculate its error term δ_h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in \text{outputs}} w_{h,k} \delta_k$$

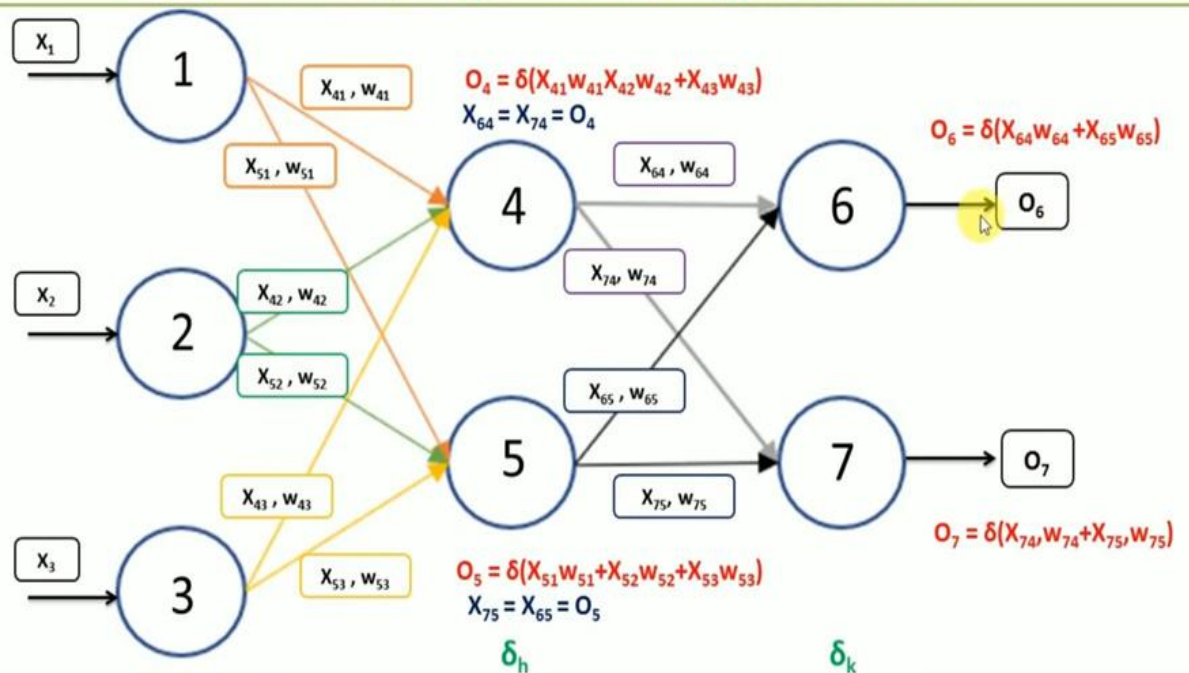
4. Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = \eta \delta_j x_{ji}$$

Back Propagation Algorithm



Derivation of Back Propagation Algorithm

- To derive the equation for updating weights in back propagation algorithm, we use Stochastic gradient descent rule.
- Stochastic gradient descent involves iterating through the training examples one at a time, for each training example $\mathbf{d}$ descending the gradient of the error E_d with respect to this single example.
- In other words, for each training example $\mathbf{d}$ every weight w_{ji} is updated by adding to it Δw_{ij} .

- That is,

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

Derivation of Back Propagation Algorithm

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

Where

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}}$$

- where E_d is the error on training example $\mathbf{d}$, that is half the squared difference between the target output and the actual output over all output units in the network,

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2$$

- Here outputs is the set of output units in the network, t_k is the target value of unit k for training example $\mathbf{d}$, and o_k is the output of unit k given training example $\mathbf{d}$.

Derivation of Back Propagation Algorithm Back Propa

Notation Used:

x_{ji} = the i^{th} input to unit j

w_{ji} = the weight associated with the i^{th} input to unit j

$net_j = \sum_i w_{ji} x_{ji}$ (the weighted sum of inputs for unit j)

o_j = the output computed by unit j

t_j = the target output for unit j

σ = the sigmoid function

outputs = the set of units in the final layer of the network

Downstream(j) = the set of units whose immediate inputs include the output of unit j

Derivation of Back Propagation Algorithm

Case 1: Training Rule for Output Unit Weights

- Just as w_{ji} can influence the rest of the network only through net_j , net_j can influence the network only through o_j . Therefore, we can invoke the chain rule again to write,

$$\begin{aligned} \frac{\partial E_d}{\partial net_j} &= \frac{\partial E_d}{\partial o_j} \frac{\partial o_j}{\partial net_j} & \frac{\partial E_d}{\partial o_j} &= \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2 & \frac{\partial o_j}{\partial (net_j)} &= \frac{\partial \sigma(net_j)}{\partial (net_j)} & \frac{\partial \sigma(x)}{\partial (x)} &= \sigma(x) (1 - \sigma(x)) \\ & & & & &= \sigma(net_j) (1 - \sigma(net_j)) \\ & & & & &= o_j (1 - o_j) \\ \frac{\partial E_d}{\partial o_j} &= \frac{\partial}{\partial o_j} \frac{1}{2} (t_j - o_j)^2 & & & & & & \\ &= \frac{1}{2} 2(t_j - o_j) \frac{\partial (t_j - o_j)}{\partial o_j} & & & & & & \\ &= -(t_j - o_j) & & & \frac{\partial E_d}{\partial net_j} &= -(t_j - o_j) o_j (1 - o_j) \end{aligned}$$

Derivation of Back Propagation Algorithm

Case 2: Training Rule for Hidden Unit Weights

$$\begin{aligned}
 \frac{\partial E_d}{\partial net_j} &= \sum_{k \in \text{Downstream}(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j} & \frac{\partial E_d}{\partial net_j} &= -(t_j - o_j) o_j (1 - o_j) \\
 &= \sum_{k \in \text{Downstream}(j)} -\delta_k \frac{\partial net_k}{\partial net_j} & \delta_j &= (t_j - o_j) o_j (1 - o_j) \\
 &= \sum_{k \in \text{Downstream}(j)} -\delta_k \frac{\partial net_k}{\partial o_j} \frac{\partial o_j}{\partial net_j} & \frac{\partial net_k}{\partial o_j} &= \frac{\partial x_{kj} w_{kj}}{\partial o_j} = \frac{\partial o_j w_{kj}}{\partial o_j} \\
 &= \sum_{k \in \text{Downstream}(j)} -\delta_k w_{kj} \frac{\partial o_j}{\partial net_j} & \frac{\partial o_j}{\partial net_j} &= \frac{\partial \sigma(net_j)}{\partial net_j} \\
 &= \sum_{k \in \text{Downstream}(j)} -\delta_k w_{kj} o_j (1 - o_j) & &= \sigma(net_j) (1 - \sigma(net_j)) \\
 & & &= o_j (1 - o_j)
 \end{aligned}$$

Derivation of Back Propagation Algorithm

Case 2: Training Rule for Hidden Unit Weights

$$\begin{aligned}
 \Delta w_{ji} &= -\eta \frac{\partial E_d}{\partial net_j} x_{ji} & \frac{\partial E_d}{\partial net_j} &= \sum_{k \in \text{Downstream}(j)} -\delta_k w_{kj} o_j (1 - o_j) \\
 \Delta w_{ji} &= \eta o_j (1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj} x_{ji} \\
 \Delta w_{ji} &= \eta \delta_j x_{ji} & \delta_j &= o_j (1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj}
 \end{aligned}$$

Challenges in Deep Learning: Overfitting, Vanishing/Exploding Gradients

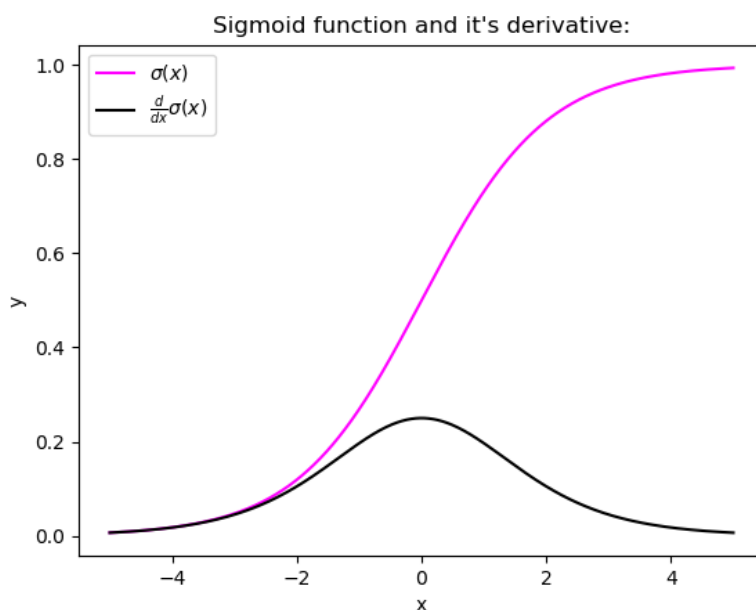
Vanishing and Exploding Gradients:

Vanishing Gradients problem:

Vanishing is when as backpropagation occurs, the gradients normally get smaller and smaller, gradually approaching zero. This leaves the weights of the initial or lower layers unchanged, causing the Gradient Descent to never converge to the optimum.

For example, Activation Functions such as the sigmoid function have a very prominent difference between the variance of their inputs and outputs. They shrink and transform a large input space into a smaller output space, which lies between $[0,1]$.

Looking at the graph below of the Sigmoid Function, we can see that using larger inputs, regardless if they are negative or positive will classify at either 0 or 1. However, when the Backpropagation processes, it has no gradient to propagate backward in the Neural Network. The little gradient that does exist, will continuously keep diluting as the algorithm continues to process through the top layers, leaving nothing for the lower layers.



Exploding Gradients problem:

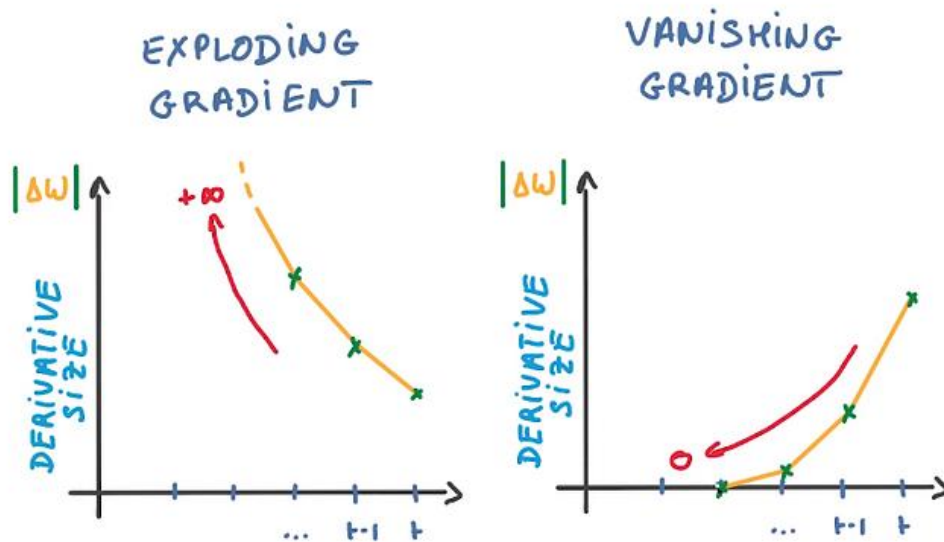
Exploding is the opposite of Vanishing and is when the gradient continues to get larger which causes a large weight update and results in the Gradient Descent to diverge.

Exploding gradients occur due to the weights in the Neural Network, not the activation function.

The gradient linked to each weight in the Neural Network is equal to a product of numbers. If this contains a product of values that is greater than one, there is a possibility that the gradients become too large.

The weights in the lower layers of the Neural Network are more likely to be affected by Exploding Gradient as their associated gradients are products of more values. This leads to the gradients of the lower layers being more unstable, causing the algorithm to diverge.

Press enter or click to view image in full size

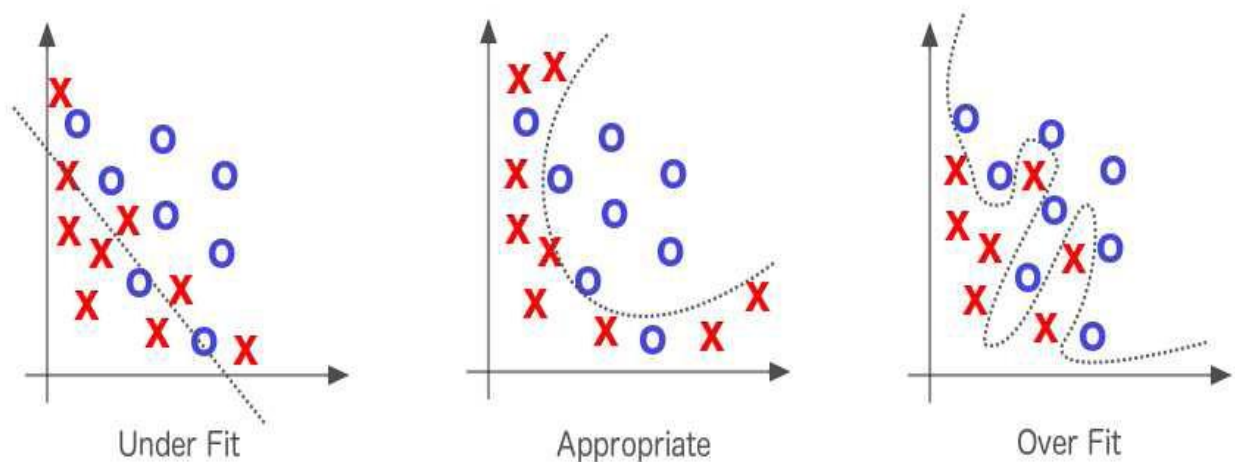


To address these problems the following techniques can be used:

- **Weights Regularizations:** The initialization of weights can be adjusted to ensure that they are in an appropriate range. Using a different activation function, such as the Rectified Linear Unit (ReLU), can also help to mitigate the vanishing gradient problem.
- **Gradient clipping:** It involves limiting the maximum and minimum values of the gradient during backpropagation. This can prevent the gradients from becoming too large or too small and can help to stabilize the training process.
- **Batch normalization:** It can also help to address these problems by normalizing the input to each layer, which can prevent the activation function from saturating and help to reduce the vanishing and exploding gradient problems.

Underfitting and Overfitting

Overfitting and Underfitting are two fundamental challenges encountered when training deep learning models, impacting their ability to generalize to unseen data.



Overfitting:

- **Definition:**

Overfitting occurs when a deep learning model learns the training data too well, including its noise and specific patterns, leading to excellent performance on the training set but poor performance on new, unseen data. The model essentially memorizes the training data rather than learning generalizable features.

- **Characteristics:**

High variance and low bias. The model is overly complex for the underlying data distribution.

- **Causes:**

- Excessive model complexity (too many layers or neurons).
- Insufficient training data relative to model complexity.
- Noisy or irrelevant features in the training data.

- **Detection:**

Large discrepancy between training error (low) and validation/test error (high).

Underfitting:

- **Definition:**

Underfitting occurs when a deep learning model is too simple to capture the underlying patterns and relationships within the training data. It fails to learn effectively from the training data, resulting in poor performance on both the training set and unseen data.

- **Characteristics:**

High bias and potentially low variance. The model is too simplistic for the complexity of the data.

- **Causes:**

- Insufficient model complexity (too few layers or neurons).
- Insufficient training time or epochs.
- Poor feature engineering or irrelevant features.

- **Detection:**

High training error and high validation/test error, indicating the model hasn't learned the data well at all.

To overcome Overfitting and Underfitting:

For Overfitting:

- **Regularization:** Techniques like L1/L2 regularization or dropout to penalize complex models and prevent reliance on specific features.
- **Early Stopping:** Monitoring validation performance and stopping training when it starts to degrade, preventing the model from memorizing noise.
- **Data Augmentation:** Increasing the effective size of the training data by creating variations of existing samples.
- **Simplifying the Model:** Reducing the number of layers or neurons if the model is excessively complex.

For Underfitting:

- **Increasing Model Complexity:** Adding more layers or neurons to allow the model to learn more intricate patterns.
- **Longer Training:** Training the model for more epochs to allow it to converge.
- **Feature Engineering:** Creating more relevant and informative features from the raw data.
- **Choosing a More Suitable Model Architecture:** Selecting an architecture that aligns better with the complexity of the problem.